



DOCUMENTO DE DISEÑO FUNCIONAL

SIMI - Diseño funcional de la interfaz y del flujo de procesamiento

VERSIÓN	FECHA	AUTOR
1.1	Junio 2026	Duarte, Christian Héctor
DOCUMENTO BASE		
ERS SIMI v1.0		

0. Historial de revisiones

Versión	Fecha	Cambios
1.0	Junio 2026	Versión inicial: diseño funcional completo, especificación de estados, widgets y comportamientos.
1.1	Junio 2026	Actualización post-implementación: tecnologías reales verificadas, pipeline de traducción en tres capas, comportamiento de reset inmediato al cargar video, corrección del enum <code>QMediaPlayer.PlaybackState</code> , dependencias definitivas.

1. Propósito y alcance del documento

Este documento traduce los requerimientos de la ERS de SIMI en un diseño funcional concreto: la estructura de la ventana principal, el comportamiento de cada control, el flujo de estados de la aplicación y la nomenclatura de los componentes de interfaz. La versión 1.1 incorpora las decisiones tomadas durante la implementación y los ajustes verificados en el entorno de ejecución real.

Cada elemento de este diseño está trazado a los requerimientos RF-01 a RF-10 y RNF-01 a RNF-06 de la ERS v1.0.

2. Arquitectura funcional

SIMI se organiza en cuatro módulos. La interfaz nunca ejecuta procesamiento pesado: delega en los módulos de servicio a través de hilos de trabajo (`QThread`) y refleja su progreso (RF-09).

Módulo	Responsabilidad	Tecnología implementada
Interfaz (GUI)	Presentación, previsualización del video, control del flujo de estados y entrada del usuario.	PySide6 6.11.1 / Qt Widgets
Extractor de audio	Extrae la pista de audio del video a WAV 16 kHz mono (formato requerido por el ASR). Requiere que <code>ffmpeg</code> esté disponible en el PATH del sistema.	FFmpeg (proceso externo vía <code>subprocess</code>)
Motor ASR	Transcribe el audio a texto detectando automáticamente el idioma. Carga el modelo en memoria la primera vez (carga perezosa) y lo reutiliza en llamadas sucesivas.	faster-whisper 1.2.1 · modelo <code>small</code> en CPU
Motor de traducción	Traduce el texto transcrito al idioma de destino seleccionado mediante un pipeline en tres capas (ver §2.1).	Argos Translate 1.11.0 (capa principal operativa)

2.1 Pipeline de traducción en tres capas

La función de traducción intenta tres métodos en orden decreciente de prioridad. Si uno falla, cae silenciosamente al siguiente:

Capa	Motor	Estado en v1.1	Nota
1	transformers 5.12 + modelos Helsinki-NLP (MarianMT)	Inactiva — incompatibilidad entre transformers 5.x y la arquitectura MarianMT para la tarea <code>translation</code> .	Capturada por <code>except Exception</code> ; la capa 2 toma el control.
2	Argos Translate 1.11.0	Activa y operativa. Paquetes instalados: <code>en→es</code> , <code>es→en</code> , <code>en→pt</code> , <code>pt→en</code> .	Si un par de idiomas no tiene paquete instalado, lo descarga automáticamente vía <code>pkg.install()</code> .
3	Texto de demostración (hardcoded)	Fallback final si las capas 1 y 2 fallan.	Devuelve frases fijas por idioma; no traduce el texto real. Solo visible si no hay conexión y no hay paquetes instalados.

2.2 Dependencias del entorno de ejecución

Paquete	Versión	Rol
Python	3.14	Intérprete base.
PySide6	6.11.1	Framework GUI (Qt 6.11).
faster-whisper	1.2.1	Motor ASR offline (CTranslate2).
argostranslate	1.11.0	Motor de traducción offline.
transformers	5.12.0	Instalado; actualmente inactivo para traducción (incompatibilidad v5.x).
torch	2.12.0	Backend de inferencia (requerido por stanza, dependencia de argostranslate).
ffmpeg	sistema	Extracción de audio. Debe estar en el PATH del sistema operativo.

3. Diseño de la pantalla principal

La aplicación tiene una **única ventana** (RNF-03: todo el flujo sin menús de configuración), dividida en cuatro zonas: **A)** previsualización, **B)** panel de proceso, **C)** paneles de texto y **D)** barra de estado y progreso.

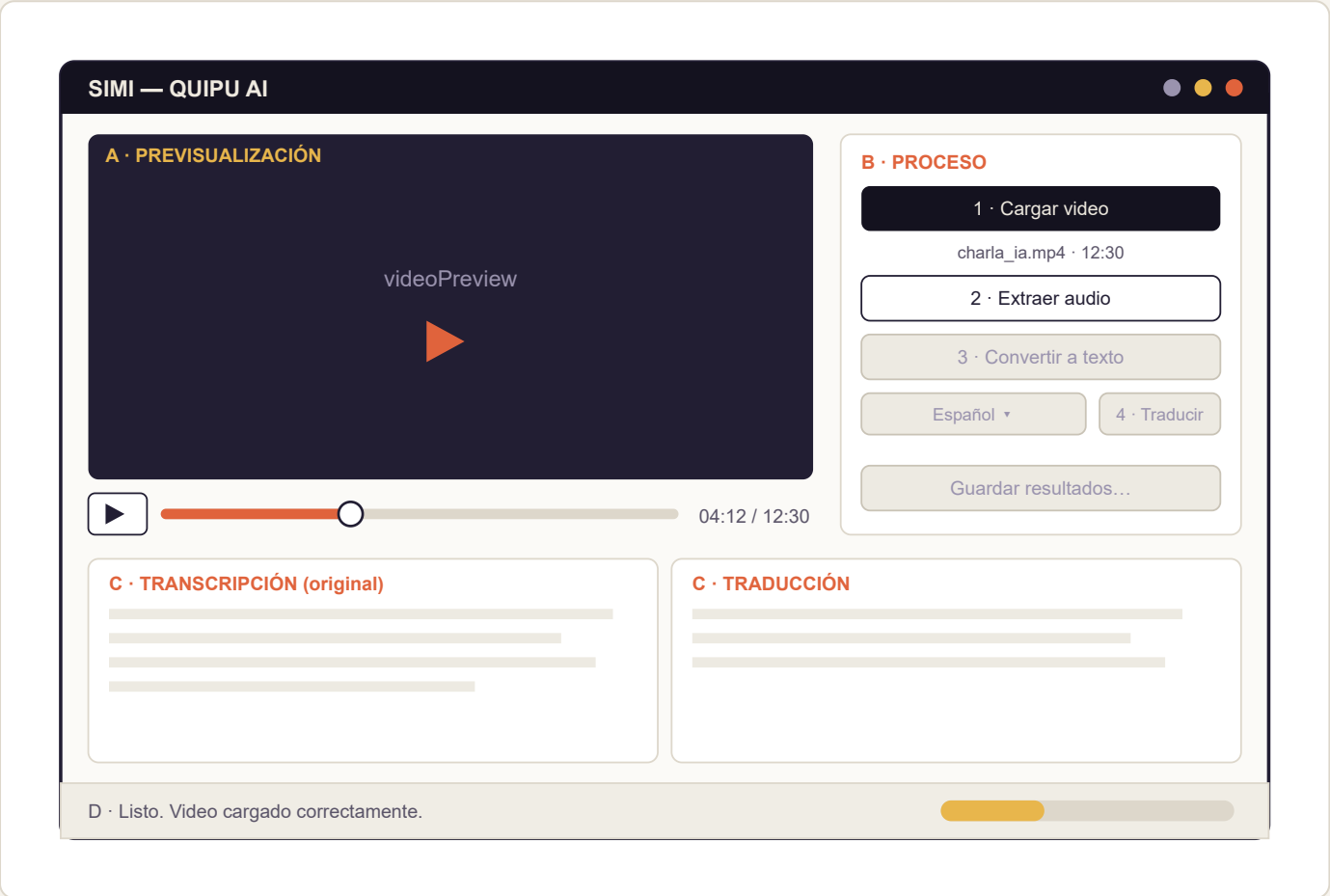


Figura 1 — Esquema de la ventana principal de SIMI (estado E1: video cargado, botón «Extraer audio» habilitado).

3.1 Zona A — Previsualización (RF-03)

- Área de reproducción de video con relación de aspecto preservada. Al cargarse un video, Qt6 ejecuta `play()` + `pause()` en secuencia para forzar el render del primer fotograma (comportamiento necesario en Qt 6 / PySide6 6.x).
- Botón de reproducir/pausar (alterna el ícono ► / II según el estado de reproducción). Utiliza el enum `QMediaPlayer.PlaybackState.PlayingState` de PySide6 6.x.
- Barra deslizadora de posición sincronizada con la reproducción; permite buscar una posición arrastrando.
- Etiqueta de tiempo con formato `mm:ss / mm:ss` (posición actual / duración total).

3.2 Zona B — Panel de proceso (RF-01, 02, 04–08, 10)

- Botones numerados 1 a 4 que reflejan el orden del flujo. La numeración comunica visualmente la secuencia (RF-10).
- Debajo del botón «Cargar video», una etiqueta muestra el nombre y la duración del archivo cargado.
- La lista de idiomas y el botón «Traducir» comparten fila: seleccionar idioma y traducir es una sola acción conceptual.
- «Guardar resultados...» abre un diálogo de archivo estándar para exportar los .txt (RF-08).

3.3 Zona C — Paneles de texto (RF-05, RF-07)

- Dos paneles lado a lado: **Transcripción** (editable: el usuario puede corregir errores del ASR antes de traducir) y **Traducción** (editable, pero pensado como solo lectura).
- Separados por un divisor ajustable para repartir el ancho según necesidad.

3.4 Zona D — Estado y progreso (RF-09, RNF-06)

- Barra de estado con mensajes en lenguaje claro: «Listo», «Extrayendo audio...», «Transcribiendo (esto puede demorar)...», «Traducción completada».
- Barra de progreso visible solo durante operaciones. Usa **modo indeterminado** (`setRange(0, 0)`) tanto en la extracción de audio (FFmpeg no reporta porcentaje) como en la transcripción (faster-whisper tampoco reporta progreso parcial).
- Los errores (RNF-06) se muestran en un cuadro de diálogo modal con el mensaje definido en la ERS y la aplicación vuelve al último estado estable.

4. Flujo de estados (RF-10)

La aplicación se comporta como una máquina de estados. Cada botón solo está habilitado cuando su precondition se cumple; durante cualquier procesamiento, todos los controles de proceso se deshabilitan (estado P).



Figura 2 — Diagrama de estados de SIMI. Las operaciones largas pasan por el estado P y regresan al estado siguiente o, ante error, al anterior.

Estado	Controles habilitados	Entrada al estado
E0	Solo «Cargar video». Paneles de texto vacíos, preview en negro.	Inicio de la aplicación, o inicio de carga de un nuevo video (reset inmediato).
E1	«Cargar video», preview y controles de reproducción, «Extraer audio».	Video válido cargado (≤ 30 min, formato admitido) y metadatos leídos por QMediaPlayer.
E2	Los de E1 + «Convertir a texto».	Audio extraído con éxito (WAV 16 kHz mono generado por FFmpeg).
E3	Los de E2 + lista de idiomas, «Traducir», «Guardar resultados...».	Transcripción mostrada en el panel original.
E4	Todos.	Traducción mostrada en el panel de traducción.
P	Ninguno (solo la reproducción del preview permanece disponible). Barra de progreso activa en modo indeterminado.	Inicio de extracción, transcripción o traducción.

Regla de invalidación — Cargar nuevo video: al pulsar «Cargar video», los paneles de texto se limpian y el estado pasa a E0 *de forma inmediata*, antes de que el reproductor cargue el archivo. Cuando QMediaPlayer confirma la duración, se aplica E1. Si se carga el mismo archivo, la fuente del reproductor se vacía primero (`setSource(QUrl())`) para forzar la recarga y el disparo de la señal `durationChanged`.

Regla de invalidación — Editar transcripción: si el usuario edita `txtTranscripcion` estando en E4, la traducción queda marcada como desactualizada, el estado regresa a E3 y el botón

5. Especificación de componentes (nomenclatura Qt)

Los nombres de objeto definidos aquí son los que se usan en el archivo

`ui/simi_mainwindow.ui` de Qt Designer y en el código Python (`main.py`). El archivo generado es `ui/simi_mainwindow_ui.py` . Convención: prefijo por tipo de control (`btn` , `lbl` , `cmb` , `txt` , `sld`) + nombre descriptivo en camelCase.

objectName	Clase Qt	Función	Traza
MainWindow	QMainWindow	Ventana principal. Título: «SIMI — QUIPU AI». Tamaño mínimo 1000×700.	—
videoPreview	QVideoWidget	Área de previsualización del video (Qt Multimedia). Renderiza el primer fotograma mediante <code>play()+pause()</code> al cargar.	RF-03
btnPlayPausa	QPushButton	Alterna reproducción/pausa. Texto: ► en pausa, II en reproducción. Compara contra <code>QMediaPlayer.PlaybackState.PlayingState</code> .	RF-03
sldPosicion	QSlider (horizontal)	Posición de reproducción; permite búsqueda por arrastre.	RF-03
lblTiempo	QLabel	Tiempo actual / duración (<code>mm:ss / mm:ss</code>).	RF-03
btnCargarVideo	QPushButton	«1 · Cargar video». Abre diálogo de archivo con filtro MP4/AVI/MKV; resetea UI inmediatamente; valida duración ≤ 30 min al recibir <code>durationChanged</code> .	RF-01, RF-02
lblArchivo	QLabel	Nombre y duración del archivo cargado. Muestra «Cargando...» mientras se leen los metadatos.	RF-01
btnExtraerAudio	QPushButton	«2 · Extraer audio». Lanza FFmpeg en hilo de trabajo. Requiere <code>ffmpeg</code> en PATH.	RF-04
btnConvertirTexto	QPushButton	«3 · Convertir a texto». Lanza faster-whisper modelo <code>small</code> en hilo de trabajo.	RF-05
cmbIdioma	QComboBox	Idiomas de destino: Español (defecto), Portugués, Inglés.	RF-06
btnTraducir	QPushButton	«4 · Traducir». Lanza la traducción al idioma de <code>cmbIdioma</code> vía pipeline de tres capas.	RF-07
btnGuardar	QPushButton	«Guardar resultados...». Exporta <code><nombre>_transcripcion.txt</code> y <code><nombre>_traduccion_<idioma>.txt</code> en UTF-8.	RF-08
txtTranscripcion	QPlainTextEdit	Texto original (editable). Cambios en E4 disparan regla de invalidación.	RF-05
txtTraduccion	QPlainTextEdit	Texto traducido.	RF-07
splitterTextos	QSplitter	Divisor ajustable entre los dos paneles de texto.	—

progressBar	<code>QProgressBar</code>	Progreso de la operación en curso. Modo indeterminado (<code>setRange(0,0)</code>) para extracción y transcripción; oculta al terminar.	RF-09
statusbar	<code>QStatusBar</code>	Mensajes de estado y errores no críticos.	RF-09, RNF-06

6. Comportamiento detallado de las acciones

Acción	Comportamiento implementado
Cargar video	1. Abre diálogo de archivo (filtro: <code>*.mp4 *.avi *.mkv</code>). 2. Inmediatamente: limpia <code>txtTranscripcion</code> y <code>txtTraduccion</code>, restablece <code>ruta_wav</code> e <code>idioma_detectado</code> a <code>None</code>, aplica estado E0 y muestra «Cargando...» en <code>lblArchivo</code>. 3. Vacía la fuente del reproductor (<code>setSource(QUrl())</code>) y luego carga el nuevo archivo para garantizar el disparo de <code>durationChanged</code> aunque sea el mismo video. 4. Cuando <code>durationChanged</code> entrega la duración: si $> 30:00$, muestra diálogo de error y permanece en E0; si es válido, actualiza <code>lblArchivo</code>, ejecuta <code>play()+pause()</code> para mostrar el primer fotograma y pasa a E1.
Extraer audio	Pasa a P → lanza FFmpeg en hilo <code>QThread</code> (<code>-vn -ac 1 -ar 16000</code> , salida WAV junto al video original) con barra en modo indeterminado → al terminar muestra «Audio extraído: <ruta>» y pasa a E2. Ante fallo de FFmpeg: diálogo de error y regreso a E1. Si FFmpeg no está en PATH: error inmediato.
Convertir a texto	Pasa a P con mensaje «Transcribiendo (esto puede demorar)...» → carga faster-whisper modelo <code>small</code> en CPU (carga perezosa: solo la primera vez) → transcribe el WAV y detecta el idioma → vuelca el texto en <code>txtTranscripcion</code> usando <code>blockSignals(True/False)</code> para no activar la regla de invalidación → guarda el idioma detectado para la capa de traducción → pasa a E3. Ante fallo: texto de fallback en inglés, idioma asumido <code>en</code> .
Traducir	Toma el contenido <i>actual</i> de <code>txtTranscripcion</code> (incluye correcciones del usuario) y el idioma de <code>cmbIdioma</code> → pasa a P → ejecuta el pipeline de tres capas (§2.1) en hilo de trabajo → muestra el resultado en <code>txtTraduccion</code> y pasa a E4. Si el idioma de destino coincide con el detectado, devuelve el texto sin modificar. El texto traducido preserva la codificación UTF-8 completa (incluyendo caracteres como ã, ê, ç para portugués).
Guardar resultados...	Diálogo «Guardar como» con nombre sugerido <code><stem_video>_transcripcion.txt</code> → genera ese archivo en UTF-8; si existe traducción, genera también <code><stem_video>_traduccion_<idioma>.txt</code> en la misma carpeta.

Concurrencia y robustez: todas las operaciones pesadas (extracción, transcripción, traducción) corren en instancias de `Trabajador(QThread)`. Si se intenta lanzar una nueva operación mientras otra está en curso, la aplicación muestra un aviso y descarta la petición. Los errores no capturados en el hilo emiten la señal `fallo(str)` que activa el diálogo de error en el hilo principal (RNF-06).

